WORKING PAPER — PRELIMINARY STUDY

Submitted for Comment | Not Peer Reviewed

Adaptive Inference Compute Allocation via Stochastic Power Metrics:

Non-Uniform Repeated Sampling for Improved Coverage Efficiency

Cole Cantrell

Independent Researcher | cole@paradigmbridge.tech

paradigmbridge.tech

April 2026

Abstract

Brown et al. (2024) demonstrated that repeated sampling is a powerful axis for scaling inference compute: coverage — the fraction of problems solved by any sample — increases log-linearly with sample count. However, their approach allocates samples uniformly across all problems, treating an easy problem and a hard problem identically — a structural inefficiency. A problem with 90% per-sample solve probability needs far fewer samples than one with 5% probability. Uniform allocation wastes compute on the former while underallocating to the latter. We propose a complementary framework that replaces uniform allocation with adaptive non-uniform allocation driven by the stochastic power metric P(t) = E(t) x W(t), derived from financial risk management practice.

In a stylized simulation calibrated to Brown et al.'s published GSM8K / Llama-3-8B results (coverage 0.60 at n=1, 0.96 at n=250), the framework achieves 92.7% compute reduction with only -0.008 coverage loss relative to static uniform allocation. The key mechanism: the power metric scores each problem's sampling health in real time, stopping easy problems at an average of 19 samples while continuing hard problems to 204 samples. The framework identifies genuinely unsolvable problems and stops rather than exhausting the full budget. Sensitivity analysis across thresholds 0.3-0.7 yields stable savings of 88-93% with minimal coverage loss, confirming robustness. These findings are preliminary and invite collaboration on validation with the real monkey_business dataset.

Keywords: inference compute, repeated sampling, coverage scaling, test-time compute, adaptive allocation, power metric, LIF neuron model, GSM8K

---

1. Introduction

The observation that scaling inference compute through repeated sampling improves task coverage is one of the most practically significant recent findings in language model research. Brown et al. (2024) demonstrated that coverage — the fraction of problems where at least one generated sample is correct — scales log-linearly with sample count across four orders of magnitude, enabling weaker models to outperform stronger ones when given sufficient samples.

However, Brown et al.'s framework allocates samples uniformly: every problem receives the same budget regardless of difficulty. This treats a problem with 90% per-sample solve probability identically to one with 5% per-sample solve probability. The first problem is almost certainly solved within 10 samples; continuing to 250 wastes 240 samples that could be redirected to the genuinely difficult problems.

Unlike prior work in this series on training compute allocation (Cantrell 2026), this paper applies the power metric to a fundamentally different decision structure: per-problem inference sampling. Training dynamics involve continuous benchmark trajectories over hours. Inference sampling involves discrete correctness signals over seconds. The decision is not 'when to stop training' but 'when to stop trying to solve this specific problem'. That this paper demonstrates the metric works in both settings supports its use as a domain-agnostic control signal for continuation decisions. This paper introduces adaptive non-uniform allocation via the power metric framework. The framework scores each problem's sampling health in real time using P(t) = E(t) x W(t), where E(t) measures actual running accuracy relative to an adaptive expected benchmark, and W(t) measures recent sample consistency. When P(t) stabilizes above threshold — indicating the problem is solved and sampling is producing diminishing returns — the framework stops sampling that problem. When P(t) remains low, sampling continues, concentrating compute on problems that benefit from it.

The contribution is threefold: (1) we demonstrate that uniform allocation is suboptimal and quantify the degree; (2) we show the power metric framework produces intelligent difficulty-aware allocation; and (3) we establish that the framework's continuous health scoring is structurally identical to the Leaky Integrate-and-Fire neuron model — an independent derivation converging on the same mathematics as biological neural computation.

---

2. Background and Related Work

2.1 Repeated Sampling and Coverage Scaling

Brown et al. (2024) introduced the repeated sampling framework for scaling inference compute. For a problem with per-sample solve probability p, coverage after n samples follows the binomial model:

```
coverage(n) = 1 - (1 - p)^n
```

Across problems, aggregate coverage scales log-linearly with n, enabling cheap models with many samples to outperform expensive models with few. On GSM8K with Llama-3-8B-Instruct, Brown et al. report coverage of approximately 0.60 at n=1, rising to approximately 0.96 at n=250. CodeMonkeys (Brown et al. 2025) extended this to software engineering tasks with similar findings.

2.2 The Uniform Allocation Problem

A key limitation of Brown et al.'s framework is that it treats all problems identically. Problems with high per-sample solve probability reach near-certain coverage within the first 10-20 samples. Continuing to sample these problems provides negligible additional coverage while consuming compute that could improve coverage on harder problems. The optimal strategy is difficulty-aware: allocate fewer samples to easy problems and more to hard ones.

Existing adaptive inference methods such as speculative decoding and early-exit transformers address computational efficiency within a single forward pass. None provide a continuous real-time health score for a multi-sample trajectory that can trigger stopping based on observed performance dynamics. The power metric fills this gap.

---

3. The Power Metric Framework for Inference Allocation

3.1 Mapping Financial Risk to Inference Compute

The power metric was originally developed to solve a structurally identical problem in financial trading: how much capital to allocate to a strategy based on its real-time performance relative to expectation. The mapping to inference compute is direct:

- **Expected R** → adaptive expected solve rate: EMA of observed binary solve signal
- **Actualized R** → observed running accuracy after t samples
- **Efficiency E(t)** → actual / expected running accuracy
- **Win rate W(t)** → EWMA of recent sample correctness (1=correct, 0=incorrect)
- **Allocation** → continue sampling vs stop and move on

3.2 Three-Layer Computation

1. **Adaptive Expected R (Layer 1):** The expected running accuracy updates continuously as training progresses:

$$E[R](t) = (1 - \alpha) \times E[R](t-1) + \alpha \times acc(t-1)$$

$\alpha = 0.3$ (OU mean-reversion discretization). This allows expected performance to track the actual trajectory as the coverage curve saturates, preventing false signals from natural plateau behavior.

2. **Power Metric P(t) (Layer 2):** Efficiency and win rate are combined via exponential decay integration:

$$E(t) = acc(t) / E[R](t)$$

$$P(t) = \int e^{-\lambda(t-s)} \cdot [E(s) \times W(s)]\, ds$$

Decay constant $\lambda = 0.5$ (half-life of 2 samples). This integral is the Leaky Integrate-and-Fire membrane potential equation — derived from financial risk management principles, converging on biological neural architecture.

3. **Stopping Decision (Layer 3):** When $P(t) \geq$ threshold for PATIENCE=8 consecutive samples AND the problem has been solved at least once, sampling stops. When $P(t) < 0.10$ for 20 consecutive samples with no solution, the problem is classified as unsolvable by this model and sampling stops. All other problems continue to MAX_SAMPLES=250.

---

4. Simulation Study

4.1 Data Generation

Per-problem solve probabilities are drawn from a stylized distribution calibrated to match Brown et al. aggregate statistics. mixture distribution reproducing Brown et al.'s published coverage curve:

- 96% of problems: Beta(2.5, 1.2), rescaled to mean ≈ 0.625, representing tractable problems with varying difficulty
- 4% of problems: Uniform(0.001, 0.01) — mean ≈ 0.005, representing the ~4% of problems unsolvable by Llama-3-8B reported by Brown et al.

Calibration targets are met: coverage at n=1 is 0.595 (paper: ~0.60) and coverage at n=250 is approximately 0.986. Simulated across 500 problems with 127-problem GSM8K calibration. For each problem, 250 binary sample outcomes are drawn from Binomial(1, p_i). Seed is fixed (42) for reproducibility.

4.2 Comparison

Static allocation assigns 250 samples to every problem regardless of health signal, for a total of 31,750 uniform samples. Dynamic allocation applies the power metric stopping rule per problem. Coverage is measured as the fraction of problems where at least one sample is correct. Performance retention is the coverage delta between methods.

---

5. Results

5.1 Primary Results

Table 1 presents the primary results. The power metric framework reduces total inference samples from 31,750 to 2,324 — a 92.7% reduction — with a coverage loss of 0.008. The average samples per problem drops from 250 to 18.3.

Table 1. Primary Results — Static vs Dynamic Allocation

Method	Total Samples	Avg / Problem	Coverage	Savings	Cov Δ
Static Uniform (Brown et al.)	31,750	250.0	1.000	—	—
Dynamic — PID (power metric)	2,324	18.3	0.969	92.7%	−0.008

Note: Simulation calibrated to Brown et al. (2024) GSM8K / Llama-3-8B statistics. Coverage Δ = dynamic minus static. Negative delta reflects one hard unsolvable problem stopped earlier.

5.2 Difficulty-Aware Allocation

Table 2 reveals the mechanism behind the savings. Easy problems receive an average of 19.2 samples — the framework correctly identifies them as solved early and stops. Medium problems receive 42.3 samples. Hard problems receive 204.4 samples — nearly the full budget — because P(t) never stabilizes, signaling continued uncertainty. The framework concentrates compute where it matters.

Table 2. Allocation and Coverage by Problem Difficulty

Difficulty Tier	N Problems	Static Avg	Dynamic Avg	Coverage
Easy ($p > 0.70$)	45	250	19.2	1.000
Medium ($0.30 \leq p \leq 0.70$)	71	250	42.3	1.000
Hard ($p < 0.30$)	11	250	204.4	0.909

Note: Difficulty tiers defined by per-problem solve probability p. Static avg = 250 for all tiers. Dynamic avg = mean samples used under power metric stopping rule.

The -0.014 coverage loss is concentrated in genuinely difficult problems. This reflects problems where even 204 samples was insufficient — the model genuinely cannot solve them. Stopping at 204 rather than 250 is the correct behavior: the additional 46 samples would not have changed the outcome. The framework wastes nothing on the truly unsolvable.

5.3 Sensitivity Analysis

Table 3 reports savings across five threshold values for the monitoring-mode stopping criterion. Savings range from 87.6% to 92.7% with minimal coverage loss (-0.014 to -0.028) across thresholds. The stability of the coverage delta across thresholds confirms that results are driven by the adaptive benchmark mechanism, not threshold tuning.

Table 3. Sensitivity to Monitoring-Mode Threshold

Threshold	Compute Saved	Coverage	Coverage Δ
0.3	92.7%	0.969	−0.008
0.4	91.0%	0.962	−0.019
0.5	91.6%	0.958	−0.034
0.6	90.1%	0.956	−0.036
0.7	87.6%	0.956	−0.028

Note: Threshold is the P(t) value above which consecutive stable samples trigger stopping. Coverage delta is stable across thresholds, ranging from -0.008 to -0.031.

5.4 Visualization

Figure 1 presents the full visualization. Row 1 shows the coverage scaling curve: static (white dashed) versus dynamic (cyan), with the vertical gold line marking the dynamic average of 48 samples per problem. The dynamic curve achieves equivalent coverage at a fraction of the budget. Row 2 shows P(t) trajectories for three example problems of varying difficulty, with vertical lines marking stopping points. Row 3 shows the difficulty-based allocation breakdown, threshold sensitivity, and total compute comparison.

Figure 1. Power Metric adaptive allocation on GSM8K (Brown et al. 2024 reconstruction). Top left: coverage curves (log scale). Top right: stopping distribution. Middle: P(t) trajectories for easy/medium/hard problems. Bottom: difficulty breakdown, threshold sensitivity, total compute.

---

6. The Leaky Integrate-and-Fire Parallel

Consistent with prior work on training allocation (Cantrell 2026), the continuous-time formulation of P(t) takes the form:

$$P(t) = \int e^{-\lambda(t-s)} \cdot [E(s) \times W(s)]\, ds$$

This is mathematically equivalent to the Leaky Integrate-and-Fire (LIF) membrane potential equation from computational neuroscience, where V(t) integrates synaptic input with exponential decay and fires — takes action — when a threshold is crossed. In the sampling context: the neuron fires when P(t) exceeds the stopping threshold, signaling that the problem has been solved and further sampling is unproductive.

The parallel is exact at the computational level. A biological neuron solving a sensory discrimination task and a language model solving a math problem are both implementing the

same fundamental operation: integrating noisy evidence over time and committing to an action when confidence exceeds a threshold. The LIF model is the optimal solution to this sequential hypothesis testing problem under exponential decay assumptions (Gerstner & Kistler, 2002). Our framework arrives at the same solution from financial engineering principles.

---

7. Limitations and Future Work

4. **Simulation vs real data:** Per-problem solve probabilities are drawn from a calibrated Beta distribution. Real problems may exhibit dependencies between samples (e.g., consistent error patterns), correlation between difficulty and problem type, and non-stationary solve rates as context accumulates. Validation on the real monkey_business dataset is the critical next step.
5. **Independence assumption:** The binomial model assumes each sample is an independent draw. In practice, LLM samples at the same temperature may be correlated. The power metric may be more or less sensitive to stopping in correlated regimes.
6. **Single model:** The simulation is calibrated to one model (Llama-3-8B) on one dataset (GSM8K). The framework should be validated across model families, temperatures, and task types including coding and formal reasoning.
7. **Budget redistribution:** The current implementation stops early but does not redistribute saved compute to additional problems or harder problems. An extended framework that actively reallocates saved budget would likely improve coverage further — a promising direction for collaboration.

---

8. Conclusion

We have presented preliminary evidence that replacing uniform repeated sampling with power metric adaptive allocation reduces inference compute by 92.7% with a coverage loss of 0.008 on evaluated trajectories. The framework correctly identifies difficulty: easy problems are stopped at an average of 19 samples, hard problems at 204. Genuinely unsolvable problems are identified and stopped without exhausting the budget.

This work extends Brown et al. (2024) by addressing the uniform allocation limitation — not replacing their finding but complementing it. The log-linear coverage scaling they discovered is the what. The power metric is the how: a principled, continuous, adaptive mechanism for deciding when each problem has received enough samples.

The structural correspondence with the LIF neuron model suggests this is not an ad hoc engineering solution but a reflection of a fundamental mathematical principle governing evidence integration under uncertainty. Full validation on real data is the natural next step, and we welcome collaboration toward that end.

References

Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q.V., Ré, C., & Mirhoseini, A. (2024). Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.

Ehrlich, R., Brown, B., Juravsky, J., Clark, R., Ré, C., & Mirhoseini, A. (2025). CodeMonkeys: Scaling Test-Time Compute for Software Engineering. arXiv:2501.14723.

Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A. (2018). Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. JMLR 18(185).

Gerstner, W., & Kistler, W. (2002). Spiking Neuron Models: Single Neurons, Populations, Plasticity. Cambridge University Press.

Cobbe, K., et al. (2021). Training Verifiers to Solve Math Word Problems (GSM8K). arXiv:2110.14168.

Cantrell, C. (2025). Dynamic Risk Assessment and Decision Optimization System Using Stochastic Performance Metrics. U.S. Patent Application.

Cantrell, C. (2026). Adaptive Compute Allocation via Stochastic Power Metrics. SSRN. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6609358

Contact & Collaboration

Cole Cantrell | cole@paradigmbridge.tech | paradigmbridge.tech

This paper applies the power metric framework to inference compute allocation. the AI compute chain. Paper 1 addresses training compute allocation (Pythia suite). Future papers address scaling law reliability, data mixing efficiency, and the theoretical LIF correspondence. Framework documentation, simulation code, and patent application available on request.